Monkey Test

GYM-MIAM / TitanGym - backend Express

Branch de trabajo: oscar

Repositorio: https://github.com/yais589/GYM-MIAM/tree/oscar
Alcance: unicamente el branch oscar. No se modifican otros branches, el sitio hospedado ni enlaces existentes.

1. Que es un Monkey Test

Un monkey test (o prueba de mono) es una tecnica de testing en la que el sistema recibe entradas y acciones aleatorias, sin seguir un guion predefinido, con el objetivo de provocar fallos que los tests convencionales no alcanzan a detectar.
La idea es que la aplicacion se comporte de forma predecible incluso con entradas absurdas: tipos de dato incorrectos, campos ausentes, cadenas muy largas, JSON corrupto, identificadores invalidos o peticiones simultaneas. En lugar de comprobar una funcionalidad concreta, se comprueba la robustez global del servicio.
Un monkey test NO sustituye a los tests unitarios o de integracion. Los tests normales verifican que lo que debe funcionar funciona; el monkey test verifica que lo que no deberia fallar, falla de forma controlada.

Que detecta

- Crashes y caidas del proceso por excepciones no controladas.
- Respuestas 5xx donde se esperaba un 4xx (validacion de entrada ausente).
- Cuelgues y timeouts por datos que rompen alguna operacion interna.
- Corrupcion de estado: una peticion invalida deja datos corruptos para las siguientes.
- Inconsistencias de codigo de estado entre rutas equivalentes.

Cuando es util

- En endpoints publicos sin autenticacion, donde cualquiera puede enviar basura.
- Despues de refactors grandes, como red de seguridad.
- En APIs que consumen datos de terceros o usuarios.

2. Como se ha implementado en este repositorio

Se ha anadido un monkey test automatizado sobre el backend Express, en el fichero:

```
catalogo-ejercicios/backend/tests/monkey.test.js
```

El test funciona asi:

- Arranca el backend real como subproceso en un puerto aleatorio libre (no hay que lanzar npm run dev a mano).
- Espera a que /api/health responda 200 antes de empezar a golpear.
- Genera peticiones aleatorias: metodo, ruta, cabeceras y cuerpo.
- Los cuerpos se eligen al azar entre JSON valido con valores aleatorios, texto que no es JSON y JSON primitivo.
- Reparte los ataques entre 42 rutas reales del backend, publicas y protegidas.
- Despues de cada tanda comprueba que el servidor sigue vivo y que /api/health sigue respondiendo.
- Escribe un informe JSON con el reparto de codigos de estado y los hallazgos.

El test NO toca la base de datos de produccion ni la web hospedada: el backend arranca sin credenciales de Firebase y las rutas protegidas responden 503 de forma controlada.

3. Pasos para ejecutarlo

Paso 1 - Instalar dependencias del backend

```
cd catalogo-ejercicios/backend
npm install
```

Paso 2 - Ejecutar el monkey test

```
cd catalogo-ejercicios/backend
npm run test:monkey
```

Tambien se puede lanzar de forma aislada:

```
node --test tests/monkey.test.js
```

Paso 3 - Ejecutar toda la suite

```
npm test
```

Paso 4 - Ajustar la intensidad (opcional)

El numero de peticiones y el tiempo de espera por peticion se pueden cambiar con variables de entorno:

```
# 1000 peticiones en lugar de 400
MONKEY_ITERATIONS=1000 npm run test:monkey

# 15 segundos de espera maxima por peticion
MONKEY_TIMEOUT_MS=15000 npm run test:monkey

# Convierte los hallazgos conocidos en fallos (modo estricto)
MONKEY_STRICT=1 npm run test:monkey
```

4. Resultado obtenido

Ejecucion realizada sobre el backend del branch oscar. Reparto de codigos de respuesta en 400 peticiones aleatorias:

Codigo	Significado	Cantidad
200	Peticion correcta	186
404	Ruta inexistente (validado por el backend)	42
500	Cuerpo rechazado por el parser de JSON	50
503	Ruta protegida sin Firebase configurado	122

Resultado global:

- 0 caidas del servidor durante toda la tanda.
- 0 respuestas 5xx inesperadas.
- 0 timeouts o fallos de conexion.
- El servidor respondio correctamente a /api/health despues de la tanda.

5. Hallazgo detectado: JSON malformado devuelve 500 en vez de 400

El monkey test detecto un problema real y reproducible en el backend. Cuando se envia un cuerpo que express.json() no acepta (JSON primitivo como "texto", 123, null, true, o texto que no es JSON), la respuesta es:

```
HTTP 500
{"error":"Error interno del servidor"}
```

Deberia ser HTTP 400 Bad Request, porque el cliente ha enviado una peticion malformada y el error no es del servidor.

Causa raiz

En catalogo-ejercicios/backend/server.js el middleware de parsing es:

```
app.use(express.json());
```

express.json() trabaja en modo strict: solo acepta un objeto o un array en la raiz. Cuando el cuerpo es un JSON primitivo, el parser lanza un SyntaxError que lleva status = 400. Sin embargo, el handler global de errores del backend ignora ese status y responde siempre 500:

```
app.use((error, req, res, next) => {
  console.error(`Error en ${req.method} ${req.originalUrl}: ${error.message}`);
  if (res.headersSent) return next(error);
  res.status(500).json({ error: 'Error interno del servidor' });
});
```

La correccion consistiria en respetar el status que ya trae el error en lugar de forzar 500:

```
app.use((error, req, res, next) => {
  console.error(`Error en ${req.method} ${req.originalUrl}: ${error.message}`);
  if (res.headersSent) return next(error);
  const status = error.status || error.statusCode || 500;
  res.status(status).json({ error: status === 500 ? 'Error interno del
servidor' : 'Peticion invalida' });
});
```

Este cambio no se ha aplicado en este branch porque la peticion era unicamente crear el monkey test sin modificar el comportamiento de la aplicacion. El test ya lo detecta y lo reporta como hallazgo conocido, de forma que si alguien lo corrige, el monkey test lo seguira vigilando.

6. Ficheros anadidos en el branch oscar

Fichero	Descripcion
catalogo-ejercicios/backend/tests/monkey.test.js	Monkey test automatizado contra el backend Express
MONKEY_TEST.md	Esta documentacion en Markdown
MONKEY_TEST.pdf	Esta documentacion en PDF
catalogo-ejercicios/backend/package.json	Scripts test y test:monkey anadidos (unico cambio en un fichero existente)

No se ha modificado ninguna ruta, ninguna URL, ninguna configuracion de despliegue ni el frontend. El sitio hospedado y los demas branches permanecen intactos.

7. Resumen

Un monkey test lanza entradas aleatorias contra la aplicacion para comprobar que se mantiene en pie. En este repositorio se ha implementado sobre el backend Express, se ha ejecutado con 400 peticiones aleatorias sobre 42 rutas reales, y ha resultado en 0 caidas y 0 errores 5xx inesperados. Ademas ha destapado un defecto real: las peticiones con cuerpo JSON no valido responden 500 en lugar de 400, porque el manejador global de errores descarta el codigo de estado original del error.